Whereas board porting involves writing custom drivers and initialisation code for devices specific to the board.

Building a Linux kernel for the target platform

Kernel building is a two-step process: first, the kernel needs to be configured for the target platform. There are many ways to configure the kernel, based on the preferred configuration interface. Given below are some of the common methods.

To run the text-based configuration, execute the following command:

```
$ make config
```

This will show the configuration options on the console as seen in Figure 1. It is a little cumbersome to configure the kernel with this, as it prompts every configuration option, in order, and doesn't allow the reversion of changes.

To run the menu-driven configuration, execute the following command:

```
$ make menuconfig
```

This will show the menu options for configuring the kernel, as seen in Figure 2. This requires the *ncurses* library to be installed on the system. This is the most popular interface used to configure the kernel.

To run the window-based configuration, execute the following command:

```
$ make xconfig
```

This allows configuration using the mouse. It requires QT to be installed on the system.

For details on other options, execute the following command in the kernel top directory:

```
$ make help
```

Once the kernel is configured, the next step is to build the kernel with the *make* command. A few commonly used commands are given below:

```
$ make vmlinux   - Builds the bare kernel
$ make modules  - Builds the modules
$ make modules_prepare  - Sets up the kernel for building the
modules external to kernel.
```

If the above commands are executed as stated, the kernel will be configured and compiled for the host system, which is generally the x86 platform. But, for porting, the intention is to configure and build the kernel for the target platform, which in turn, requires configuration of *makefile*. Two things that need to be changed in the *makefile* are given below:



Figure 1: Plain text-based kernel configuration

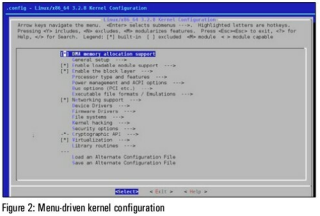


Figure 2: Menu-driven kernel configuration

```
ARCH=<architecture>
CROSS-COMPILE = <toolchain prefix>
```

The first line defines the architecture the kernel needs to be built for, and the second line defines the cross compilation toolchain prefix. So, if the architecture is ARM and the toolchain is say, from CodeSourcery, then it would be:

```
ARCH=arm
CROSS_COMPILE=arm-none-linux-gnueabi-
```

Optionally, *make* can be invoked as shown below:

```
$ make ARCH=arm menuconfig   - For configuring the kernel
$ make ARCH=arm CROSS_COMPILE=arm-none-linux-gnueabi- - For
compiling the kernel
```

The kernel image generated after the compilation is usually vmlinux, which is in ELF format. This image can't be used directly with embedded system bootloaders such as u-boot. So convert it into the format suitable for a second stage bootloader. Conversion is a two-step process and is done with the following commands:

```
arm-none-linux-gnueabi-objcopy -O binary vmlinux vmlinux.bin
mkimage -A arm -O linux -T kernel -C none -a 0x80008000 -e
0x80008000 -n linux-3.2.8 -d vmlinux.bin uImage
-A ==> set architecture
-O ==> set operating system
```